THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
Faculty of Advanced Science and Technology

PROJECT REPORT

DESIGN OF EMBEDDED SYSTEMS AND IOT

Supervisor : Nguyen Huynh Nhat Thuong

Group : 2

Class : 21ECE

Students : Vo Van Buu

Luong Nhu Quynh

Tran Hoang Minh

Nguyen Thi Tam

Da Nang, December10th 2025

Contents

1. OVERVIEW

1.1. Problem Statement

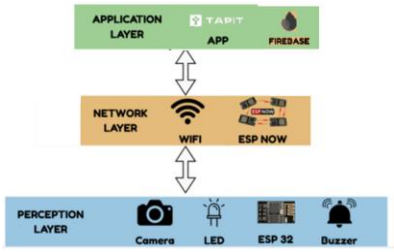
Rapid urbanization has led to severe traffic congestion, with inefficient parking management being a critical challenge. Drivers often spend excessive time searching for available spots, leading to fuel wastage and increased emissions. To address this issue, this project focuses on designing and implementing an **embedded IoT-based Smart Parking System**.

The system integrates sensors, microcontrollers, and cloud computing to facilitate real-time monitoring and management. Key functionalities include:

- **Automated Detection:** Identifying vehicle presence in specific slots.
- **Real-time Visualization:** Displaying status via LED indicators and a web interface.
- **IoT Connectivity:** Transmitting data to a central server for remote analysis.
- **Efficiency Improvement:** Optimizing parking space utilization to reduce congestion and improve the user's experience.

2. SYSTEM DESIGN

2.1. IoT Architecture



2.2. Hardware Block Diagram

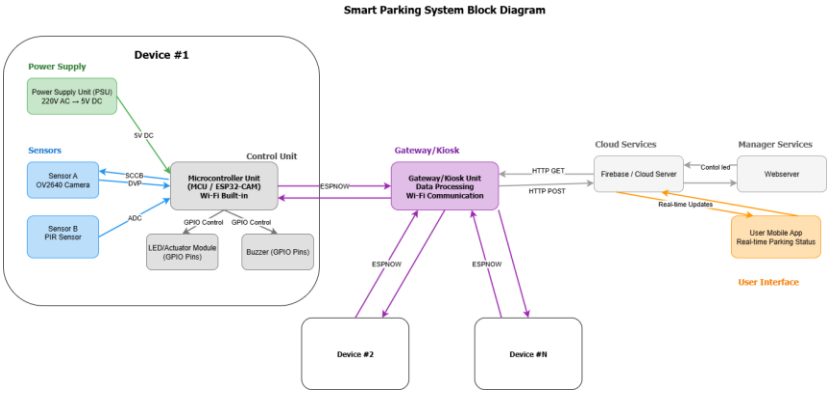
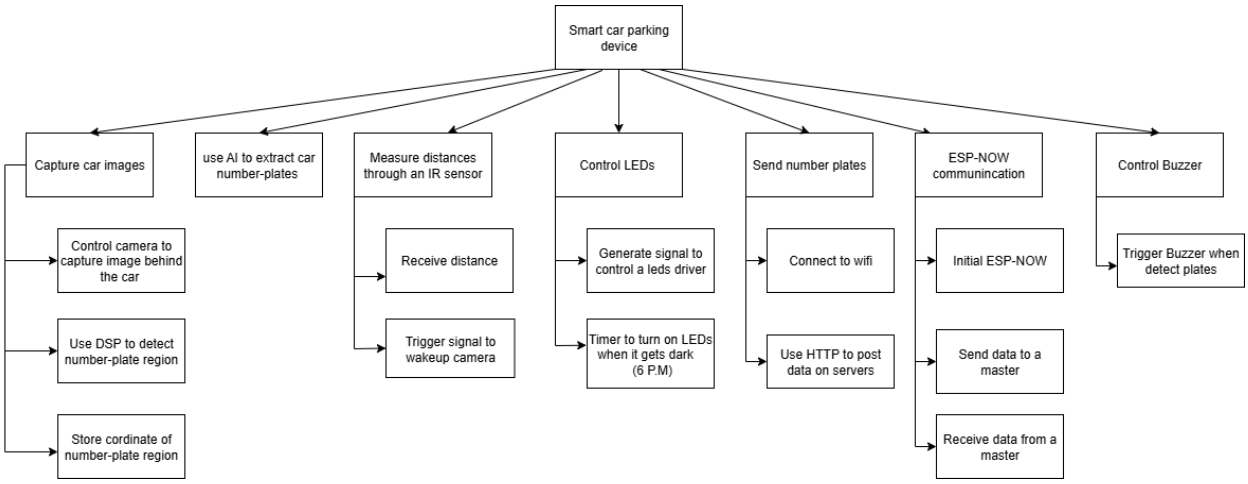


Figure : Smart Parking System Block Diagram

The system is designed based on a **Star Topology** model, utilizing a central device (Gateway) to collect data from individual parking spots. The detailed connection diagram is illustrated in Figure 1, comprising four main functional blocks:

2.2.1. Functional decomposition:



2.2.2. End Devices / Sensor Nodes:

- **Function:** Installed at each specific parking slot (Device #1, #2... #N).
- **Microcontroller:** Utilizes the **ESP32-CAM** as the central processing unit.
- **Sensors & Actuators:**
  - *ESP32 Camera:* Captures license plate images upon system wake-up.
  - *PIR Sensor:* Detects vehicle entry/exit to trigger the system.
  - *LED & Buzzer:* Provides on-site visual status indication and audible alerts.
- **Communication Protocol:** Employs the **ESP-NOW** protocol to transmit image and status data to the Gateway. This is a low-power, low-latency wireless communication protocol that operates independently of the local Wi-Fi router.

2.2.3. Central Gateway Unit (Gateway/Kiosk):

- **Function:** Acts as a **Bridge** between the local sensor network and the Internet.
- **Microcontroller:** Utilizes a separate **ESP32** module acting as the receiver.
- **Operation:**
  - Receives data packets from child nodes via the **ESP-NOW** protocol.
  - Processes and packages the data, then connects to the Internet via **Wi-Fi (HTTP protocol).**
  - Sends HTTP POST/PATCH requests to the Cloud Server and retrieves HTTP GET control commands.

2.2.4. Cloud Services:

- Uses **Firebase Realtime Database** as the central storage backend.
- Stores license plate information, parking spot status (Occupied/Available), and booking status (Reserved) in real-time.

2.2.5. Manager and User Interface:

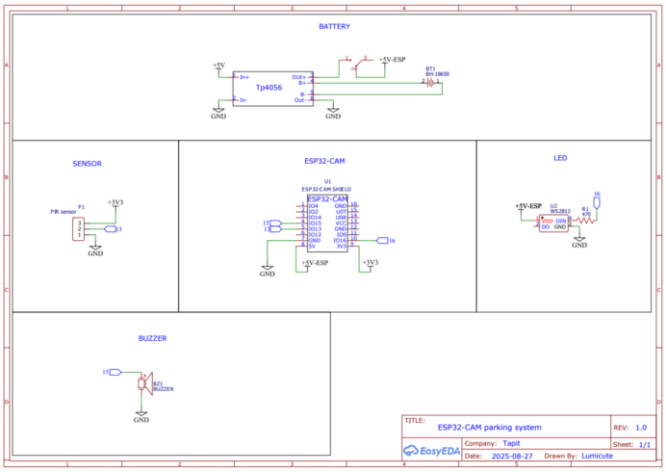
- **Web Server:** An administration dashboard allowing Admins to monitor the entire parking lot status.
- **User Mobile App:** A mobile application assisting end-users in searching for available spots and receiving real-time updates from the Cloud.

2.3. Schematic and Wiring diagram

2.3.1. Bill of Materials

No.	Component Name	Quantity	Price (đồng)	Total Price (đồng)	Note
1	Adapter	1	40	40	Available
2	TP 4056	1	5	5	Available
3	18650 battery	1	20	20	Available
4	ESP 32 CAM OV 2640	1	180	180	Available
5	PIR Sensor	1	60	60	
6	ESP 32	1	79	79	Available
7	LED RGB	1	5	5	
8	Case	1	50	50	Available
Total				439	

2.3.2. Schematic diagram



2.3.3. Wiring diagram:

Figure. ESP32-cam after integrating the camera.

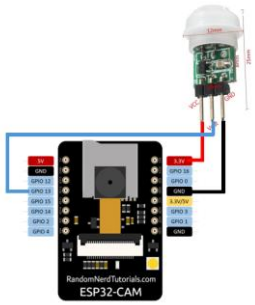


Figure. Wiring diagram for PIR sensor.

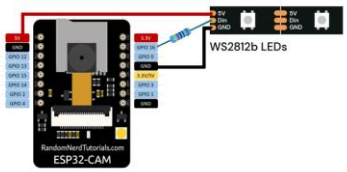


Figure. Wiring diagram for WS2812b LEDs.

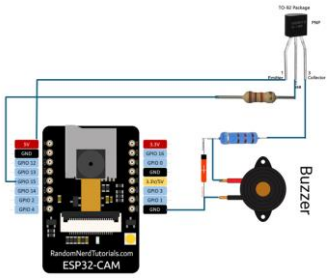


Figure. Wiring diagram for buzzer.

2.4. Software design

2.4.1. Software layer

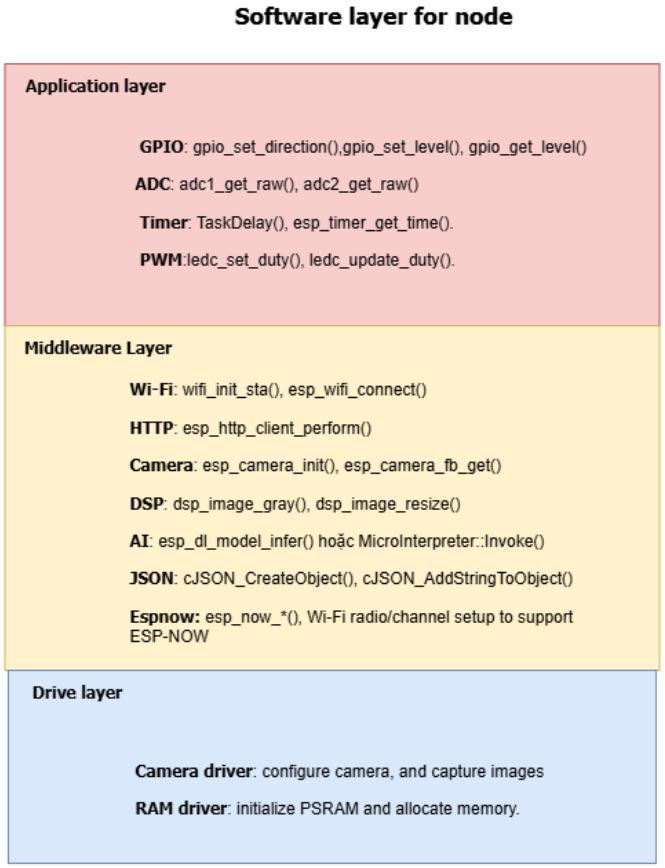


Figure : Software layer

3. COMMUNICATION & ALGORITHMS

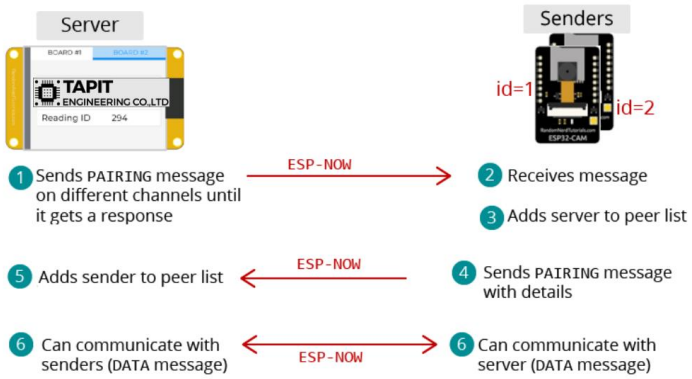
The smart parking management system utilizes Wi-Fi 5 (IEEE 802.11ac) as its primary wireless communication standard. Wi-Fi 5 operates on the 5 GHz frequency band, providing higher data transfer speeds and reduced signal interference compared to the 2.4 GHz band. This standard supports channel bandwidths of 20, 40, 80, and up to 160 MHz, allowing a theoretical data rate of up to 6.9 Gbps with eight MIMO (Multiple Input Multiple Output) streams. The system also employs 256-QAM modulation to enhance data throughput and reliability.

3.2. Protocols

3.2.1. ESP NOW

ESP-NOW is a fast, low-power wireless communication protocol developed by Espressif for ESP32 and ESP8266 microcontrollers. It enables devices to communicate directly with each other (peer-to-peer) without needing a Wi-Fi router or an internet connection.

- Direct Connection
- Speed & Efficiency
- Range: (up to 200+ meters in open areas)
- Payload Limit: Up to 250 bytes of data
- Versatility: one-way and two-way communication setups.



3.2.2. HTTP PATCH

When the camera and PIR sensor detect a vehicle, ESP32-CAM captures the vehicle's license plate number and sends this information to the cloud via an HTTP PATCH request. This request updates the corresponding parking slot record to indicate that the slot is occupied or reserved, without replacing the entire dataset.

HTTP request structure:

```
PATCH /api/spot/1 HTTP/1.1
Host: example.com
Authorization: Bearer <access_token>
Content-Type: application/json
```

HTTP response structure:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
{
 "license_plate": "51A-12345",
  "spot_id": 1
}
```

3.2.3. HTTP GET

HTTP GET is used to retrieve the current status of the LED indicator associated with each parking slot. The LED has two color states: green and red. The **green light** signals that the spot is available when there is no car. Conversely, the **red light** means a car is already parked. Additionally, when a user reserves the parking spot on the app, the light at that location will also turn **red** to indicate the spot has been reserved.

HTTP request structure:

```
GET /api/led/status?id=1 HTTP/1.1
Host: example.com
Authorization: Bearer <access_token>
Content-Type: application/json
```

HTTP response structure:

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "led_id": 1,
```

```
    "status": "green"
}
```

3.3. Set up client/server

3.3.1. Sending the License Plate Number to the Server

In our project, the distance sensor detects the movement, and then it triggers the camera to wake up and take a photo. The photo is preprocessed and passed through the AI model to extract letters and combine them into a license plate number. ESP Kiosk acts as a client, utilizing the HTTP protocol with a PATCH to send a license plate number to the Firebase Realtime Database.

Set up for server:

- Go to the **Firebase Console → Realtime Database →** Create a database (choose a region, for example, asia-southeast1).
- Get your **database URL**, for example: https://parking-violation-app-default-rtdb.asia-southeast1.firebasedatabase.app/
- **Grant write permissions:** Go to the **Rules** tab and temporarily set both .read and .write to true
- Create a field with the name licensePlate to receive the license Plate number from ESP.

Set up for Client:

- ConFigure credentials for Firebase

The code sets the Firebase API key and database URL, then performs an anonymous sign-up:
conFigure.api_key = API_KEY;
conFigure.database_url = DATABASE_URL;

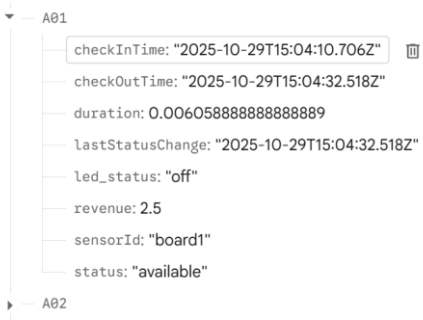
- Connect to Wi-Fi: Call WiFi. begin(ssid, password) and wait until WiFi.status() == WL_CONNECTED.
- Generate a random license plate: Example: 93W-848.78
- Send the license plate to Firebase
- Firebase.RTDB.setString(&fbdo, licensePlatePath.c_str(), licensePlate.c_str());
- Using the function seString of Firebase libray which using Patch methode to update data.
- Error handling & cleanup
- If sign-up fails, the error message from Firebase is shown.
- If writing to RTDB fails, `fbdo.errorReason()` is printed.
- Wi-Fi reconnection is handled automatically by Firebase.

3.3.2. Get the LED status from the server.

We also allow users to control an LED at their parking slot to express the status of the slot. Users can turn on the switch to red to express that parking is not allowed for other cars to park, even if the owner does not park at this time, so anyone

Set up for the server. ‘

In a real-time database of Firebase, we create a field with the name led_status that the user can change to a value in this field in a web/app application to control the LED of a device.



Set up for Client

- Include required libraries

```
#include <WiFi.h>
```

#include <Firebase_ESP_Client.h>
#include <Adafruit_NeoPixel.h> // Provide the token generation process info. #include "addons/TokenHelper.h" // Provide the real-time database payload printing
#include "addons/RTDBHelper.h"

These handle:

- Wi-Fi connection
- Secure HTTPS (TLS)
- Firebase Realtime Database operations
- JSON parsing and object mapping

- ConFigure credentials
  - Wifi name and password
  - The API Key of Firebase
  - The database URL: "https://esp-project-5cd9d-default-rtdb.asia-southeast1.firebasedatabase.app/"
  - Pins on the ESP to control the LED.
  - This authenticates the ESP32 as a Firebase user (via email/password authentication).
- Initialize Wi-Fi

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.print(".");
}
Serial.println(WiFi.localIP());
```

- Initialise Firebase
  - initializeApp(aClient, app, getAuth(user_auth), processData, "authTask");
  - app.getApp<RealtimeDatabase>(Database);
  - Database.url(DATABASE_URL);
  - This:
  - Authenticates with Firebase using user credentials.
  - Connects to the specified Realtime Database.
  - Registers a callback (`processData`) to handle database events.
- Listen for real-time database updates (GET status continuously)

- Use ArduinoJson or simple string matching to check if "ledstatus" is "ON" or "OFF."

  Then set the LED pin accordingly:

```
if (payload.indexOf("\"ledstatus\":\"ON\"") != -1) {
  digitalWrite(LED_BUILTIN, HIGH);
} else {
  digitalWrite(LED_BUILTIN, LOW); }
```

- Error handling and retry

  If httpCode <= 0, handle it as a connection failure.

  Optionally add retry logic (e.g., 3 attempts with a 1-second delay).

  Always call https.end() after each request.

3.4. Flowcharts

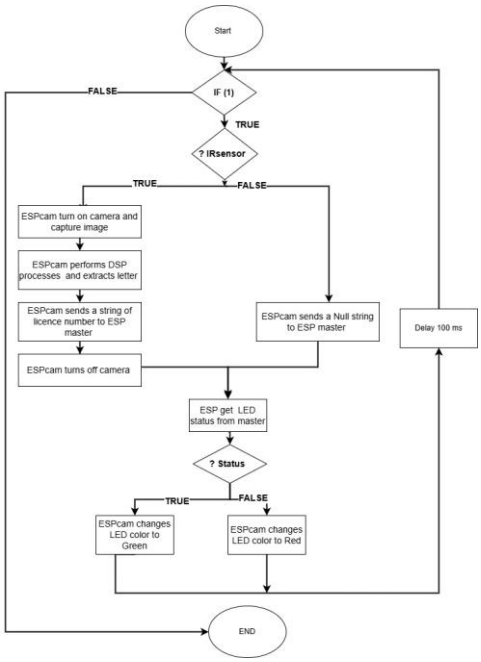


Figure: Operating flowchart for node devices.

3.4.1. PIR sensor

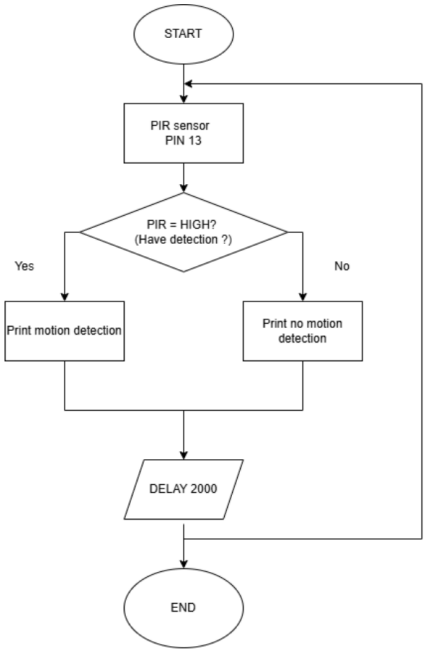


Figure 6. Flow chart of PIR sensor

3.4.2. Esp cam

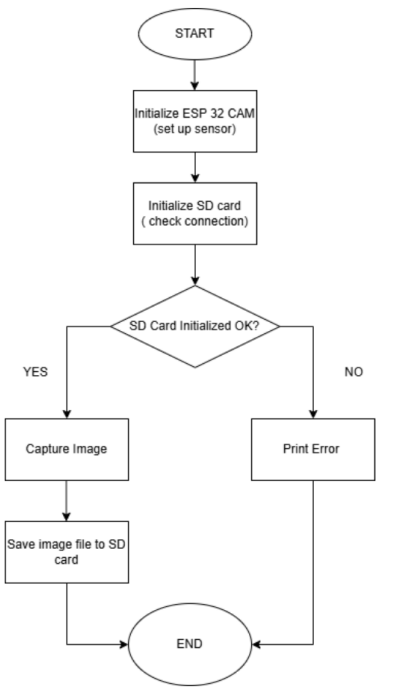


Figure 7. An algorithm for capturing an image and storing it in an SD card.

3.4.3. LED

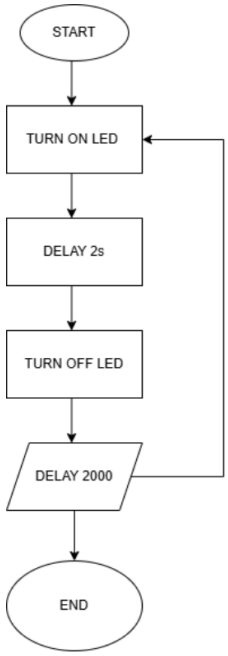


Figure 9. Flow chart Blink LED 2 seconds

3.4.4. Buzzer

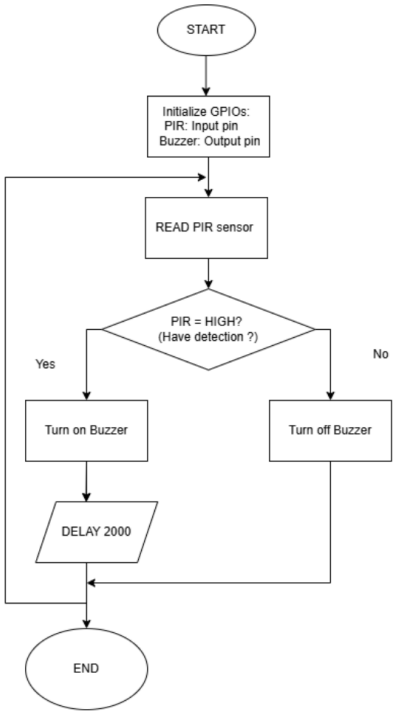


Figure 10. Buzzer with PIR sensor

3.5. Sequence Diagram

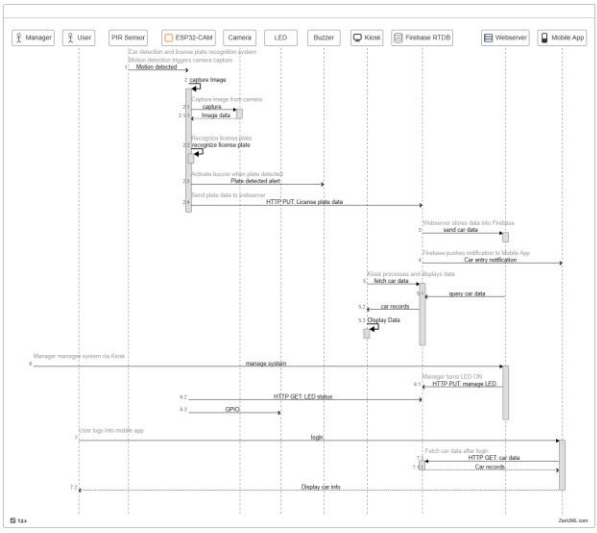


Figure : Sequence diagram

4. AI MODEL:

We use the convolutional Neural Network (CNN) because it has excellent support from many frameworks such as TensorFlow, PyTorch, TFLite, and Edge Impulse, so the deployment process is straightforward. Another important reason is speed and deployment. CNN is lightweight, fast, and easy to run on mobile or embedded devices. To maximize this potential, we leverage Edge Impulse to streamline the entire pipeline from data ingestion to deploying a highly optimized TinyML model. By utilizing its EON Tuner, we can automatically navigate the trade-off between accuracy and latency, ensuring the CNN architecture is perfectly tailored to the strict resource constraints of our hardware

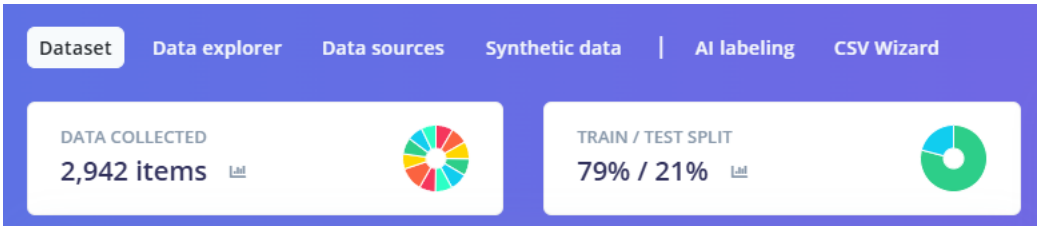


Figure. Dataset split automatically into train set (79%) and test set (21%).

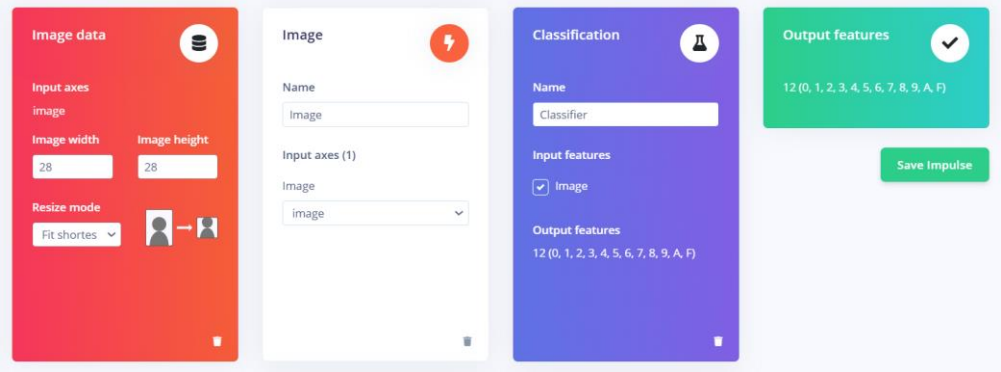


Figure. The impulse design and the output feature including 12 classes

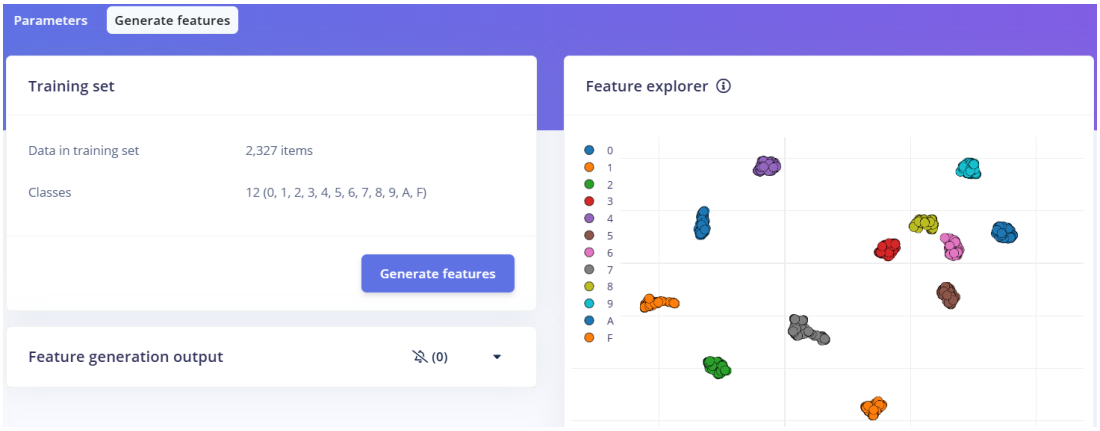


Figure. Feature extraction of 12 characters from 0 to 9, A and F.

Figure . Accuracy of the model and the confusion matrix.

Quantized (int8)

Selected ✔

	IMAGE	CLASSIFIER	TOTAL
LATENCY	7 ms.	47 ms.	**54 ms.**
RAM	4.0K	18.0K	**18.0K**
FLASH	-	52.5K	-
ACCURACY			**99.49%**

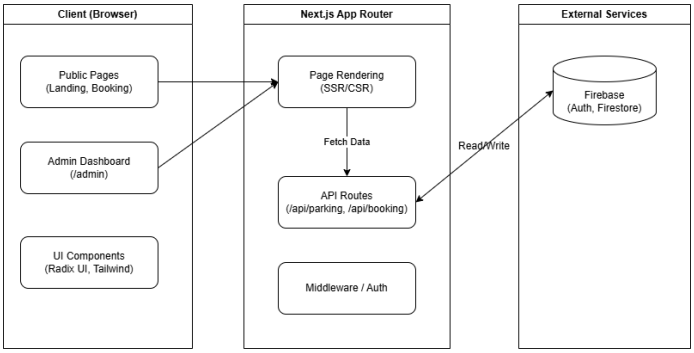
Figure . Model after quantization (int8).

5. WEB & MOBILE

5.1. Technology Stack

The "SmartPark" software system is designed to provide an intuitive interface for both end-users and administrators. To meet the requirement for real-time responsiveness, the team selected the following technology stack:

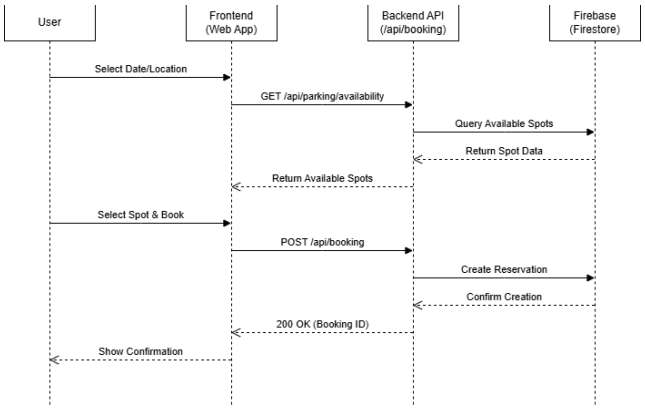
- **Framework: Next.js 14 (App Router)** - Ensures high performance and SEO optimization.
- **Language: TypeScript** - Enhances code reliability and maintainability.
- **Styling: Tailwind CSS**, combined with **Radix UI** and **Lucide React** for a modern UI design.
- **Backend & Database: Firebase Realtime Database** - Serves as the core for synchronizing states between the hardware (ESP32 Gateway) and the Web Application.
- **Form Management:** React Hook Form and Zod (Validation).



5.2. Key Functions

5.2.1. Parking Monitoring

- **Function:** Tracks the status of each parking slot/zone in real-time.
- **Operation:** Visualizes status via color codes on the map:
  - **Available (Green):** Empty spot.
  - **Occupied (Red):** Vehicle detected by sensors or pre-booked via app.
- **Sequence diagram:**



5.2.2. Reservation System

- **User:** Search for available spots, select time slots (Start Time - End Time), and receive booking confirmation.
- **Admin:** View, cancel, or adjust reservations.
- **Payment Simulation:** Calculates parking fees based on duration and simulates the checkout process.

5.2.3. Device Control

- Allows Admins (or authorized Users) to control the LED indicator at specific slots via the Web/App interface (by updating the `led_status` field on Firebase).

5.3. Architecture & Processing Logic

5.3.1. Booking Process When a user books a spot on the Web App:

- **Frontend:** Sends a `POST` request with details (License Plate, Name, Phone, Time) to the `/api/parking` API Route.
- **Server Processing:**
  - Generates a unique Booking ID.
  - Calculates estimated fees.
  - Locates the first available spot in the selected zone.
- **Database Update:**
  - Saves details to the `bookings` node.
  - Updates the slot status to `Reserved` and `led_status` to `Red` on Firebase.
- **Hardware Sync:** The Gateway receives the signal and turns the physical LED red to indicate the spot is taken.

5.3.2. Synchronization Logic

The system prioritizes the "Reserved" status to ensure customer rights:

- If **Vehicle Present (Sensor = 1):** Status is `Occupied` (Red).
- If **No Vehicle (Sensor = 0)** BUT **Booking Exists (Booking = True):** Status is `Reserved` (Purple/Red).
- If **No Vehicle** AND **No Booking:** Status is `Available` (Green).

5.4. User Interface & Demo

5.4.1. User Application Interface:

The user interface focuses on simplicity to help drivers book spots quickly.

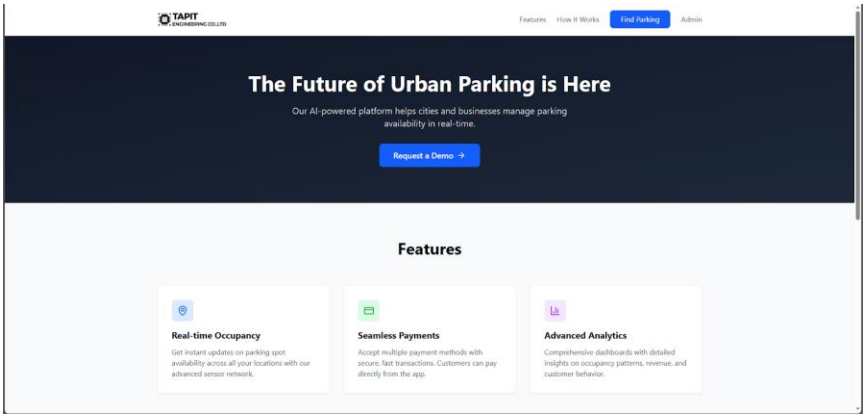


Figure: The main home page for the user to book a slot.

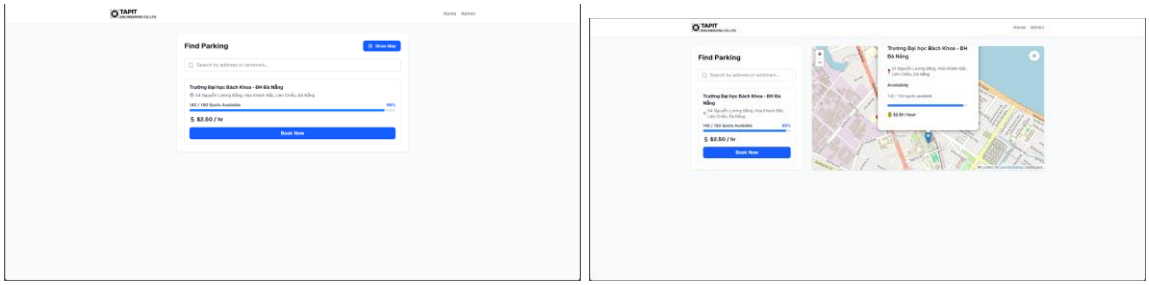


Figure: Searching for a parking slot.

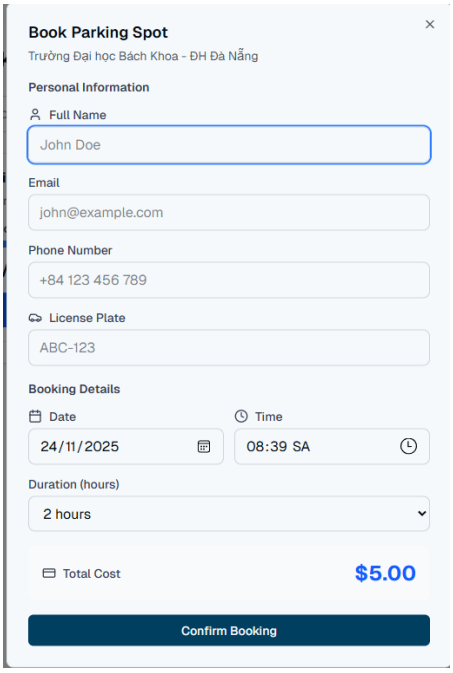


Figure: Register form for booking users.

5.4.2. Admin Dashboard

The dashboard provides a comprehensive view for operations management.

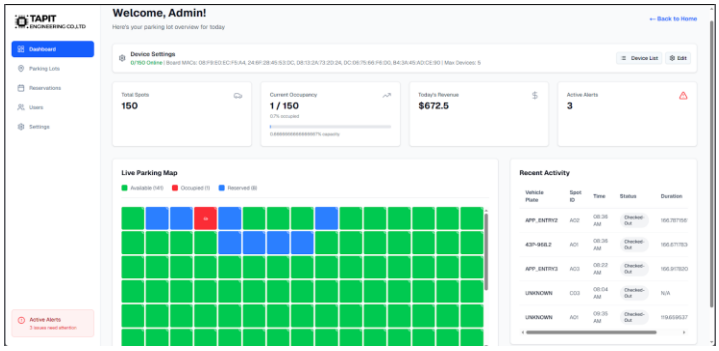


Figure: Monitoring dashboard page for admin.

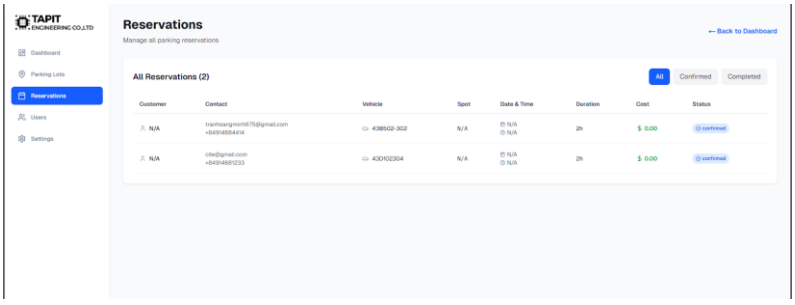


Figure: A Reserving booking page.

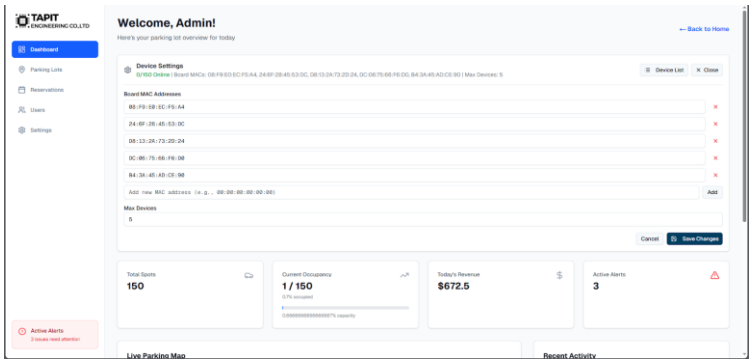


Figure: The administrator page for device conFigureuration.

5.4.3. Experimental Results

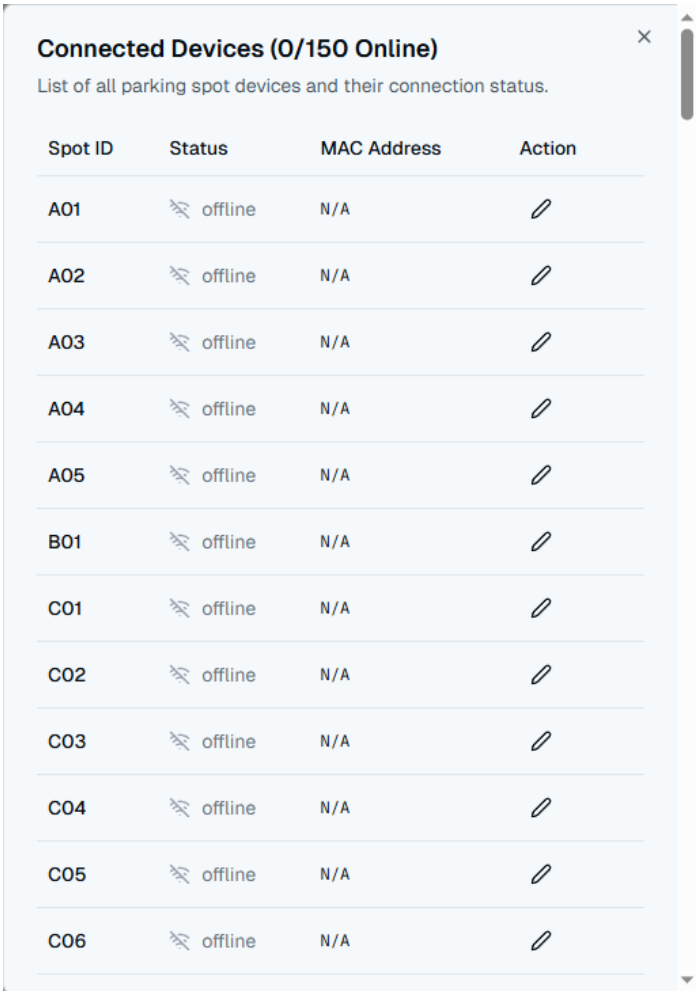


Figure: App display screen of connected devices for users.

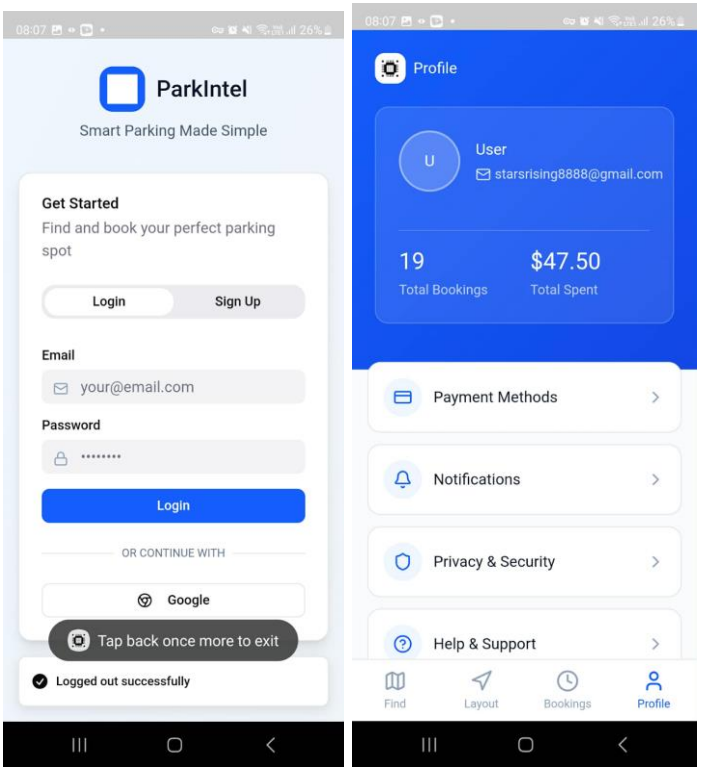


Figure: Main page and functional pages.

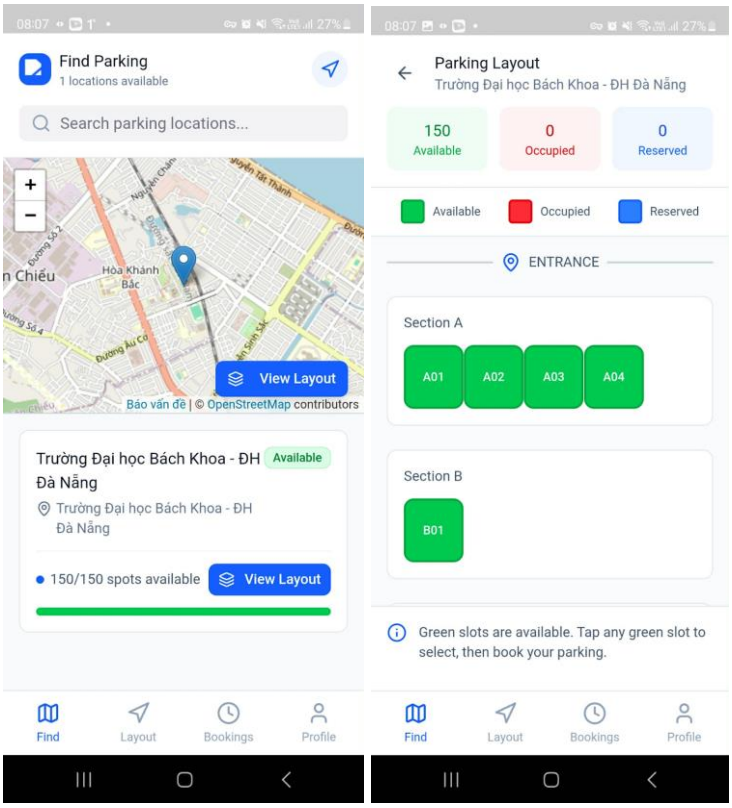


Figure: Register form for booking users in user app

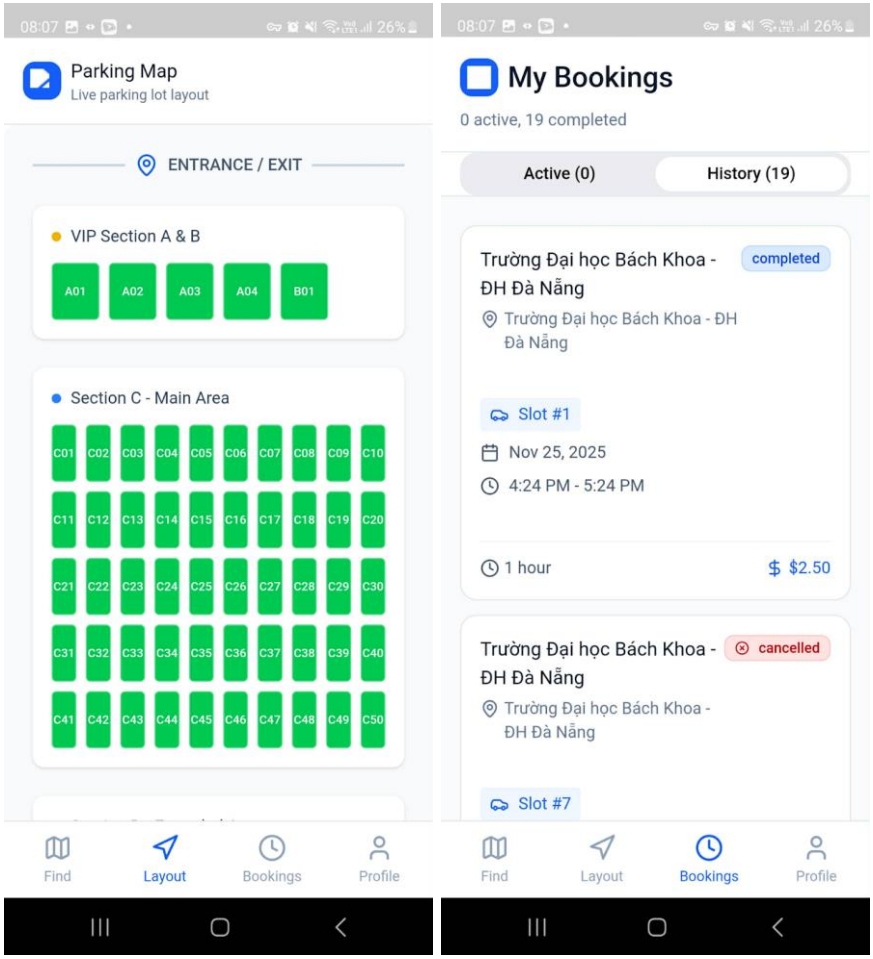


Figure: Monitoring dashboard page

6. RESULT

6.1. 3d prototype

6.2. Result at Edge

Step 1: Capture an image with the view in front of a car with a size of 640*480.

Figure: Image in front of a car captured by an ESP cam.

Step 2: Using the DSP algorithm to find and crop the licence plate number region.

Figure: Extracted licence plate after DSP processing

Step 3: Using the DSP algorithm to crop and resize to 28*28 for each letter.

Figure: Extracted number after resizing

Step 4: Load the AI model and normalise the letter to a suitable form of AI model input, and use the AI to find the license plate number.

```
Xử lý ký tự 1: X[31-63] (Kích thước: 33x68)
  ✅ Nhận dạng: 4 (độ tin cậy: 0.996)

Xử lý ký tự 2: X[75-108] (Kích thước: 34x68)
  ✅ Nhận dạng: 3 (độ tin cậy: 0.996)

Xử lý ký tự 3: X[118-148] (Kích thước: 31x68)
  ✅ Nhận dạng: A (độ tin cậy: 0.996)

Xử lý ký tự 4: X[183-215] (Kích thước: 33x68)
  ✅ Nhận dạng: 4 (độ tin cậy: 0.996)

Xử lý ký tự 5: X[225-258] (Kích thước: 34x69)
  ✅ Nhận dạng: 2 (độ tin cậy: 0.996)

Xử lý ký tự 6: X[268-302] (Kích thước: 35x69)
  ✅ Nhận dạng: 3 (độ tin cậy: 0.996)

Xử lý ký tự 7: X[324-358] (Kích thước: 35x69)
  ✅ Nhận dạng: 5 (độ tin cậy: 0.996)

Xử lý ký tự 8: X[367-395] (Kích thước: 29x70)
  ✅ Nhận dạng: 4 (độ tin cậy: 0.938)

🎯 === KẾT QUẢ NHẬN DẠNG: 43A42354 ===
```

Figure: Result of prediction

Step 5: Sending the licence plate to the Database

```
🎯 === KẾT QUẢ NHẬN DẠNG: 43A42354 ===

🎯 BIỂN SỐ NHẬN DẠNG: 43A42354
📤 Gửi dữ liệu lên Firebase: {"licensePlate":"43A42354"}
✅ Gửi Firebase thành công, mã: 200
📨 Phản hồi: {"licensePlate":"43A42354"}
```

```
📸 IMAGE URL: https://buu.pythonanywhere.com/static/uploads/plate_43A42354_20251129_183727.jpg
🎯 Direct link: https://buu.pythonanywhere.com/static/uploads/plate_43A42354_20251129_183727.jpg
📁 Filename: plate_43A42354_20251129_183727.jpg
🚗 Plate: 43A42354
🚀 Gửi lên Firebase: Biển số=43A42354, URL ảnh=https://buu.pythonanywhere.com/static/uploads/plate_43A42354_20251129_183727.jpg
📤 Gửi dữ liệu lên Firebase: {"licensePlate":"43A42354","URL_image":"https://buu.pythonanywhere.com/static/uploads/plate_43A42354_20251129_183727.jpg"}
✅ Gửi Firebase thành công, mã: 200
📨 Phản hồi: {"licensePlate":"43A42354","URL_image":"https://buu.pythonanywhere.com/static/uploads/plate_43A42354_20251129_183727.jpg"}
✅ Done processing!
```

7. TASK ADVISION

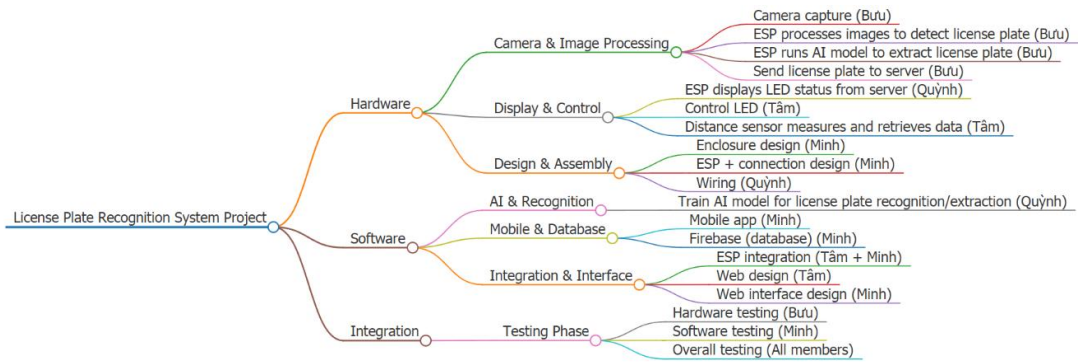


Figure: Tasks division.

8. CONCLUSION & FUTUER WORK

8.1. Conclusion

This project successfully designed and implemented an "Embedded IoT-based Smart Parking System" to address the critical issue of traffic congestion caused by inefficient parking management. By integrating embedded systems, wireless communication, and cloud computing, the system offers a comprehensive solution for real-time parking monitoring.

The key achievements of the project include:

- **System Architecture:** Successfully constructed a Star Topology network where multiple Sensor Nodes (End Devices) communicate with a central Gateway.
- **Communication Protocol:** Effectively utilized the **ESP-NOW** protocol for communication between the Sensor Nodes and the Gateway. This approach ensured low latency and high energy efficiency compared to traditional Wi-Fi connections for individual nodes.
- **Hardware Integration:** The hardware design robustly integrates **ESP32-CAM**, PIR sensors, and visual indicators (LEDs/Buzzers) to detect vehicle presence and capture license plate images accurately upon system wake-up.
- **Cloud & User Interface:** Established a reliable connection with the **Firebase Realtime Database**, allowing for seamless data synchronization. The developed Web Interface and Mobile Application provide administrators and users with intuitive, real-time access to parking status.

8.2. Limitations

Despite the successful implementation and operation of the prototype, the current system encounters certain limitations that need to be addressed in practical large-scale deployments:

Image Processing Accuracy: The license plate recognition capability of the ESP32-CAM is currently sensitive to environmental factors, particularly lighting conditions. Low-light environments significantly reduce recognition accuracy.

Internet Dependency: The system's remote monitoring capability relies entirely on the Gateway's internet connection. Any disruption in the local Wi-Fi network at the Gateway results in a loss of synchronization with the Cloud.

Power Consumption: While ESP-NOW is power-efficient, the continuous operation of the Gateway and the periodic wake-up of camera modules still require a stable power source, which limits deployment in off-grid locations.

8.3. Future Development

To enhance the system's performance, scalability, and commercial viability, the following improvements are proposed for future development:

Advanced AI Integration: Implementing Deep Learning algorithms (e.g., CNN) either on the Edge (using more powerful edge AI chips) or on the Cloud server to improve license plate recognition accuracy under various weather and lighting conditions.

E-Payment System Integration: Expanding the Mobile App features to include automatic billing and e-payment options (QR Code, E-wallets), creating a fully automated parking experience.

Energy Harvesting: Researching and integrating solar power and battery management systems (BMS) for the Sensor Nodes to enable truly wireless, maintenance-free operation in outdoor environments.

Mesh Networking: Exploring Mesh Topology (using ESP-MESH) to extend the range and reliability of the sensor network, allowing coverage for large-scale parking lots with hundreds of slots.

9. REFERENCE

[A] Academic Papers & Journals

1. T. Lin, H. Rivano, and F. Le Mouël, "A Survey of Smart Parking Solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229-3253, Dec. 2017.
2. A. Khanna and R. Anand, "IoT based smart parking system," in *2016 International Conference on Internet of Things and Applications (IOTA)*, Pune, India, 2016, pp. 266-270.
3. S. V. Srikanth, P. J. Pramod, K. P. Dileep, S. Tapas, M. U. Patil, and S. C. N. Babu, "Design and implementation of a prototype Smart Parking (SPARK) system using wireless sensor networks," in *2009 International Conference on Advanced Information Networking and Applications Workshops*, Bradford, UK, 2009, pp. 401-406.

4. M. M. Rashid, A. Musa, M. Ataur Rahman, and N. Farahana, "Automatic Parking Management System and Parking Fee Collection Based on Number Plate Recognition," *International Journal of Machine Learning and Computing*, vol. 2, no. 2, pp. 93-98, 2012.

[B] Hardware Datasheets & Manuals

5. Espressif Systems, "ESP32 Series Datasheet," Version 4.3, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
6. Ai-Thinker, "ESP32-CAM Development Board Specification," Version 1.0, 2019.
7. Adafruit Industries, "PIR Motion Sensor (HC-SR501) Datasheet," [Online]. Available: https://cdn-learn.adafruit.com/downloads/pdf/pir-passive-infrared-proximity-motion-sensor.pdf.

[C] Communication Protocols & Cloud

8. Espressif Systems, "ESP-NOW User Guide," Espressif IoT Development Framework (ESP-IDF), 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html. 9. Google Developers, "Firebase Realtime Database Documentation," [Online]. Available: https://firebase.google.com/docs/database. 10. R. N. Murthy and K. L. P. Kumar, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645-1660, 2013.